TRABAJO PRÁCTICO FINAL

Programación II

Sistema de Asistencia y Organización de Clases

Estructura de paquetes y atributos de las clases

Documento de diseño · Noviembre de 2026

1. Objetivo del documento

Este documento define la estructura de paquetes, las clases del sistema y los atributos que debe tener cada una. El objetivo es que todo el equipo implemente sobre la misma base, evitando nombres distintos, responsabilidades mezcladas y retrabajo cuando se conecte la persistencia con JDBC.

El sistema será una aplicación Java para consola, sin login y sin conexión a Internet. Los datos se guardarán localmente mediante JDBC. La aplicación permitirá administrar materias, comisiones, alumnos, ayudantes, horarios, clases, asistencias, materiales, recordatorios y copias de seguridad.

> **Criterio general:** el paquete `modelo` solo contiene datos y comportamiento propio de las entidades; no sabe guardar ni leer de la base. Las reglas de negocio viven en `servicio`. La persistencia vive en `persistencia`. El menú solo muestra opciones y delega en los servicios.

2. Estructura de paquetes

```
src/
├── Main.java
├── modelo/
│   ├── EntidadBase.java          (abstracta)
│   ├── Persona.java              (abstracta)
│   ├── Alumno.java
│   ├── Ayudante.java
│   ├── Materia.java
│   ├── Comision.java
│   ├── Establecimiento.java
│   ├── Horario.java
│   ├── EventoAgenda.java         (abstracta)
│   ├── Clase.java
│   ├── Asistencia.java
│   ├── Material.java
│   ├── Recordatorio.java
│   ├── EstadoAsistencia.java     (enum)
│   └── DiaSemana.java            (enum)
├── persistencia/
│   ├── ConexionDB.java
│   ├── Repositorio.java          (interfaz genérica)
│   ├── RepositorioMemoria.java   (genérica)
│   ├── RepositorioJDBC.java      (genérica)
```

```
│   ├── MateriaDAO.java
│   ├── ComisionDAO.java
│   ├── AlumnoDAO.java
│   ├── ClaseDAO.java
│   └── RecordatorioDAO.java
├── servicio/
│   ├── MateriaService.java
│   ├── AsistenciaService.java
│   ├── RecordatorioService.java
│   └── BackupService.java
├── excepcion/
│   ├── ValidacionException.java
│   ├── PersistenciaException.java
│   └── EntidadNoEncontradaException.java
├── menu/
│   ├── MenuPrincipal.java
│   ├── MenuMaterias.java
│   ├── MenuAsistencia.java
│   ├── MenuRecordatorios.java
│   └── Verificador.java
└── util/
    ├── Validador.java
    └── FechaUtil.java
```

`modelo`	Entidades del sistema, herencia, enums y reglas propias de cada objeto.
`persistencia`	Interfaz genérica de repositorio, implementación en memoria, conexión y DAO con JDBC.
`servicio`	Reglas de negocio, validaciones, coordinación entre repositorios y operaciones de backup.
`excepcion`	Errores personalizados de validación, persistencia y búsqueda.
`menu`	Interacción por consola. No contiene reglas de negocio.
`util`	Funciones auxiliares de validación y fechas.

3. Jerarquía de clases del modelo

```
EntidadBase (abstracta)
├── Persona (abstracta)
│   ├── Alumno
│   └── Ayudante
├── Materia
├── Comision
├── Establecimiento
├── Horario
├── EventoAgenda (abstracta)
```

```
│   ├── Clase
│   └── Recordatorio
└── Material
```

Asistencia: no hereda de EntidadBase. Es un objeto de valor que vive dentro de Map<Alumno, Asistencia>. No tiene id propio.
EstadoAsistencia y DiaSemana: enumeraciones.

4. Atributos de las clases del modelo

4.1 Entidades base y personas

EntidadBase abstracta	—	`private int id`	Base de todas las entidades persistidas. `id = 0` significa entidad nueva.

Persona abstracta	extends EntidadBase	`private String nombre`
		`private String apellido`
		`private String dni`
		`private String email`
		`private String telefono`
			Base común de `Alumno` y `Ayudante`.

Alumno	extends Persona	`private String legajo` `private List<Comision> comisiones`	Un alumno puede pertenecer a varias comisiones.

Ayudante	extends Persona	`private String area`
		`private Alumno alumnoVinculado`
		`private List<Comision> comisiones`
			`alumnoVinculado` puede ser `null`. Si existe, se reutilizan sus datos.

4.2 Entidades académicas

Materia	extends EntidadBase	`private String nombre`
		`private String codigo`
		`private List<Comision> comisiones`
			Una materia tiene una o más comisiones. El código es único.

Comision	extends EntidadBase	`private String nombre` `private Materia materia`	Agrupa alumnos, horarios y clases. Pertenece a una

		`private Establecimiento establecimiento`
		`private List<Alumno> alumnos`
		`private List<Horario> horarios`
		`private List<Clase> clases`
			materia.

Establecimiento	extends `EntidadBase`	`private String nombre`
		`private String direccion`
		`private String localidad`
			Lugar donde se dictan las clases. Puede asociarse a una comisión o a un horario.

Horario	extends `EntidadBase`	`private DiaSemana dia`
		`private LocalTime horaInicio`
		`private LocalTime horaFin`
		`private Establecimiento establecimiento`
			Horario habitual de cursada. Se valida que `horaInicio < horaFin`.

4.3 Agenda, clases y asistencia

EventoAgenda
abstracta
	extends `EntidadBase`	`private LocalDateTime fechaHora`
		`private String titulo`
		`private String descripcion`
			Base común de `Clase` y `Recordatorio`.

Clase	extends `EventoAgenda`	`private Comision comision`
		`private String tema`
		`private Map<Alumno, Asistencia> asistencias`
		`private List<Material> materiales`
			Al crearse, copia los alumnos actuales de la comisión con estado `SIN_REGISTRAR`.

Asistencia	—	`private EstadoAsistencia estado` `private String observacion`	Objeto de valor dentro del mapa de `Clase`. La clave del mapa es el `Alumno`.

Material	extends EntidadBase	`private String nombre`
		`private String descripcion`
		`private boolean llevado`
			Representa algo que se necesita para una clase.

Recordatorio	extends EventoAgenda	`private Clase clase`
		`private Material material`
		`private boolean realizado`
			`clase` y `material` pueden ser `null`. No es obligatorio asociarlo.

4.4 Enumeraciones

EstadoAsistencia	enum	`SIN_REGISTRAR`, `PRESENTE`, `AUSENTE`, `TARDE`, `JUSTIFICADA`
DiaSemana	enum	`LUNES`, `MARTES`, `MIERCOLES`, `JUEVES`, `VIERNES`, `SABADO`, `DOMINGO`

5. Métodos y responsabilidades principales

EntidadBase	`getId()`, `setId(int)`, `esNueva()`, `equals()`, `hashCode()`, `toString()`
Persona	Getters y setters de nombre, apellido, dni, email y teléfono. Constructores `protected`.
Alumno	`getLegajo()`, `setLegajo()`, `getComisiones()`, `agregarComision()`, `quitarComision()`
Ayudante	`getArea()`, `setArea()`, `getAlumnoVinculado()`, `setAlumnoVinculado()`, `getComisiones()`
Materia	`getNombre()`, `setNombre()`, `getCodigo()`, `setCodigo()`, `getComisiones()`, `agregarComision()`
Comision	Getters y setters; `agregarAlumno()`, `quitarAlumno()`, `agregarHorario()`, `agregarClase()`
Establecimiento	Getters y setters de nombre, dirección y localidad.
Horario	Getters y setters; `validarHorario()` para comprobar que el inicio sea anterior al fin.

EventoAgenda	`getFechaHora()`, `setFechaHora()`, `getTitulo()`, `setTitulo()`, `getDescripcion()`, `setDescripcion()`
Clase	`inicializarAsistencias(List<Alumno>)`, `marcarAsistencia(Alumno, EstadoAsistencia, String)`, `getEstado(Alumno)`, `agregarMaterial()`
Asistencia	`getEstado()`, `setEstado()`, `getObservacion()`, `setObservacion()`
Material	Getters y setters; `isLlevado()`, `setLlevado(boolean)`
Recordatorio	`getClase()`, `setClase()`, `getMaterial()`, `setMaterial()`, `isRealizado()`, `marcarRealizado()`

6. Capa de persistencia

ConexionDB	clase	`private static Connection conexion` `private static final String RUTA_DB`	Abrir y cerrar la conexión JDBC con SQLite. Método `getConexion()`.
Repositorio<T extends EntidadBase>	interfaz genérica	Sin atributos.	`listar()`, `buscarPorId(int)`, `guardar(T)`, `actualizar(T)`, `eliminar(int)`.
RepositorioMemoria<T extends EntidadBase>	clase genérica	`private Map<Integer, T> datos` `private int ultimoId`	Implementación en memoria para programar y probar sin base de datos.
RepositorioJDBC<T extends EntidadBase>	clase genérica abstracta	`protected Connection conexion`	Base común de los DAO. Define el mapeo de `ResultSet` y la carga de parámetros.
MateriaDAO	clase	Hereda de `RepositorioJDBC<Materia>`.	CRUD de materias.
ComisionDAO	clase	Hereda de `RepositorioJDBC<Comision>`.	CRUD de comisiones y relación con alumnos y horarios.
AlumnoDAO	clase	Hereda de `RepositorioJDBC<Alumno>`.	CRUD de alumnos y asociación con comisiones.

ClaseDAO	clase	Hereda de `RepositorioJDBC<Clase>`.	CRUD de clases, asistencias y materiales asociados.
RecordatorioDAO	clase	Hereda de `RepositorioJDBC<Recordatorio>`.	CRUD de recordatorios.

> Las entidades que dependen de otra, como `Horario`, `Material` y `Asistencia`, se guardan desde el DAO de su entidad contenedora (`ComisionDAO` o `ClaseDAO`). Así se evita duplicar operaciones y se mantiene la integridad de las relaciones.

7. Capa de servicios

MateriaService	`private Repositorio<Materia> materias` `private Repositorio<Comision> comisiones`	Crear materias y comisiones, listar, buscar y validar datos académicos.
AsistenciaService	`private Repositorio<Clase> clases` `private Repositorio<Alumno> alumnos`	Crear clases, inicializar asistencias, marcar, corregir y consultar estados.
RecordatorioService	`private Repositorio<Recordatorio> recordatorios`	Crear recordatorios, listar pendientes, obtener vencidos y marcar como realizados.
BackupService	`private String rutaBaseDatos` `private String rutaBackup`	Crear, validar y restaurar copias de seguridad locales con confirmación previa.

8. Excepciones, menú y utilidades

ValidacionException	Hereda el mensaje de	Errores de datos inválidos: DNI, nombres,

	`Exception`.	horarios, campos vacíos.
PersistenciaException	Hereda el mensaje de `Exception`.	Errores al guardar, consultar, actualizar o eliminar datos.
EntidadNoEncontradaException	Hereda el mensaje de `Exception`.	Errores cuando se busca una entidad que no existe.
MenuPrincipal	Sin atributos propios; recibe los servicios.	Mostrar el menú principal y derivar a los submenús.
MenuMaterias	Sin atributos propios; recibe `MateriaService`.	Opciones de materias, comisiones y horarios.
MenuAsistencia	Sin atributos propios; recibe `AsistenciaService`.	Opciones de clases y toma de asistencia.
MenuRecordatorios	Sin atributos propios; recibe `RecordatorioServic e`.	Opciones de recordatorios y materiales.
Verificador	`private static Scanner scanner`	Lectura y validación de entrada por consola.
Validador	Sin atributos; métodos estáticos.	Validar DNI, nombres, textos, números y horarios.
FechaUtil	Sin atributos; métodos estáticos.	Convertir y formatear fechas y horas.
Main	Sin atributos.	Crear las dependencias, instanciar servicios y arrancar el menú.

9. Relaciones principales

Materia — Comisión	1 a muchos	`List<Comision>` en `Materia` y referencia `Materia` en `Comision`.
Comisión — Alumno	muchos a muchos	`List<Alumno>` en `Comision` y tabla intermedia `comision_alumno`.
Comisión — Horario	1 a muchos	`List<Horario>` en `Comision`.

Comisión — Clase	1 a muchos	`List<Clase>` en `Comision`.
Clase — Alumno / Asistencia	1 a muchos	`Map<Alumno, Asistencia>` en `Clase`.
Clase — Material	1 a muchos	`List<Material>` en `Clase`.
Ayudante — Alumno	0 o 1	`Alumno alumnoVinculado` puede ser `null`.
Recordatorio — Clase / Material	0 o 1 cada una	Referencias opcionales en `Recordatorio`.
EventoAgenda — Clase / Recordatorio	herencia	`Clase` y `Recordatorio` extienden `EventoAgenda`.

10. Reglas de negocio y decisiones de diseño

- El DNI debe tener 8 dígitos y ser único.
- El código de materia debe ser único.
- En un horario, `horaInicio` debe ser menor que `horaFin`.
- Una comisión pertenece a una materia y puede tener varios horarios.
- Un alumno puede pertenecer a varias comisiones.
- Al crear una clase, se copian los alumnos actuales de la comisión con estado `SIN_REGISTRAR`.
- Los cambios posteriores en la comisión no modifican las clases ni las asistencias pasadas.
- Solo puede existir una asistencia por alumno y por clase.
- Corregir una asistencia modifica el estado existente; no crea otro registro.
- No se puede marcar asistencia a un alumno que no pertenece al mapa de esa clase.
- Si un ayudante está vinculado a un alumno, se reutilizan sus datos y no se duplican.
- Un recordatorio puede estar asociado a una clase, a un material, a ambos o a ninguno.
- La restauración de una copia de seguridad debe pedir confirmación antes de reemplazar datos.

> **Importante:** como `hashCode()` depende del id, una entidad no debe usarse como clave de un `Map` antes de ser guardada. Primero se guarda, el repositorio asigna el id y después se la agrega a la comisión o al mapa de asistencias.

11. Tablas de la base de datos

`materias`	`id`, `nombre`, `codigo`
`comisiones`	`id`, `id_materia`, `id_establecimiento`, `nombre`
`alumnos`	`id`, `nombre`, `apellido`, `dni`, `email`, `telefono`, `legajo`
`ayudantes`	`id`, `nombre`, `apellido`, `dni`, `email`, `telefono`, `area`, `id_alumno`

comision_alumno	id_comision, id_alumno
comision_ayudante	id_comision, id_ayudante
establecimientos	id, nombre, direccion, localidad
horarios	id, id_comision, id_establecimiento, dia, hora_inicio, hora_fin
clases	id, id_comision, fecha_hora, titulo, descripcion, tema
asistencias	id_clase, id_alumno, estado, observacion
materiales	id, id_clase, nombre, descripcion, llevado
recordatorios	id, fecha_hora, titulo, descripcion, realizado, id_clase, id_material

12. Orden de implementación

1. Modelo completo: `EntidadBase`, `Persona`, entidades, enums y constructores.
2. Excepciones personalizadas.
3. Interfaz `Repositorio<T>` y `RepositorioMemoria<T>`.
4. Servicios con reglas de negocio usando el repositorio en memoria.
5. Menú por consola reutilizando `Verificador`.
6. `ConexionDB`, `RepositorioJDBC` y DAO concretos.
7. Cambiar el repositorio en memoria por los DAO en `Main` sin tocar los servicios.
8. Backup, recordatorios, diagrama UML, documentación y commits.

Anexo A. Código base de EntidadBase

```
package modelo;

import java.util.Objects;

public abstract class EntidadBase {

    private int id;

    protected EntidadBase() {
        this(0);
    }

    protected EntidadBase(int id) {
```

```
        this.id = id;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public boolean esNueva() {
        return id == 0;
    }

    @Override
    public boolean equals(Object objeto) {
        if (this == objeto) {
            return true;
        }
        if (objeto == null || getClass() != objeto.getClass()) {
            return false;
        }
        EntidadBase otra = (EntidadBase) objeto;
        return id != 0 && id == otra.id;
    }

    @Override
    public int hashCode() {
        return id == 0 ? System.identityHashCode(this) : Objects.hash(getClass(), id);
    }

    @Override
    public String toString() {
        return getClass().getSimpleName() + "{id=" + id + "}";
    }
}
```

Anexo B. Interfaz Repositorio

```
package persistencia;

import modelo.EntidadBase;

import java.util.List;

public interface Repositorio<T extends EntidadBase> {

    List<T> listar() throws PersistenciaException;

    T buscarPorId(int id) throws PersistenciaException, EntidadNoEncontradaException;
```

```
    void guardar(T entidad) throws PersistenciaException;

    void actualizar(T entidad) throws PersistenciaException;

    void eliminar(int id) throws PersistenciaException;
}
```

Documento de diseño del Trabajo Práctico Final de Programación II. Las decisiones aquí definidas deben mantenerse durante todo el desarrollo para evitar cambios de estructura entre integrantes.